Chapter 9. Ownership Election

The previous patterns that we have seen have been about distributing requests in order to scale requests per second, the state being served, or the time to process a request. This final chapter on multi-node serving patterns is about how you scale assignment. In many different systems, there is a notion of *ownership* where a specific process owns a specific task. We have previously seen this in the context of sharded and hot-sharded systems where specific instances owned specific sections of the sharded key space.

In the context of a single server, ownership is generally straightforward to achieve because there is only a single application that is establishing ownership, and it can use well-established in-process locks to ensure that only a single actor owns a particular shard or context. However, restricting ownership to a single application limits scalability, since the task can't be replicated, and reliability, since if the task fails, it is unavailable for a period of time. Consequently, when ownership is required in your system, you need to develop a distributed system for establishing ownership.

A general diagram of distributed ownership is shown in Figure 9-1. In the diagram, there are three replicas that could be the owner or master. Initially, the first replica is the master. Then that replica fails, and replica number three then becomes the master. Finally, replica number one recovers and returns to the group, but replica three remains as the master/owner.

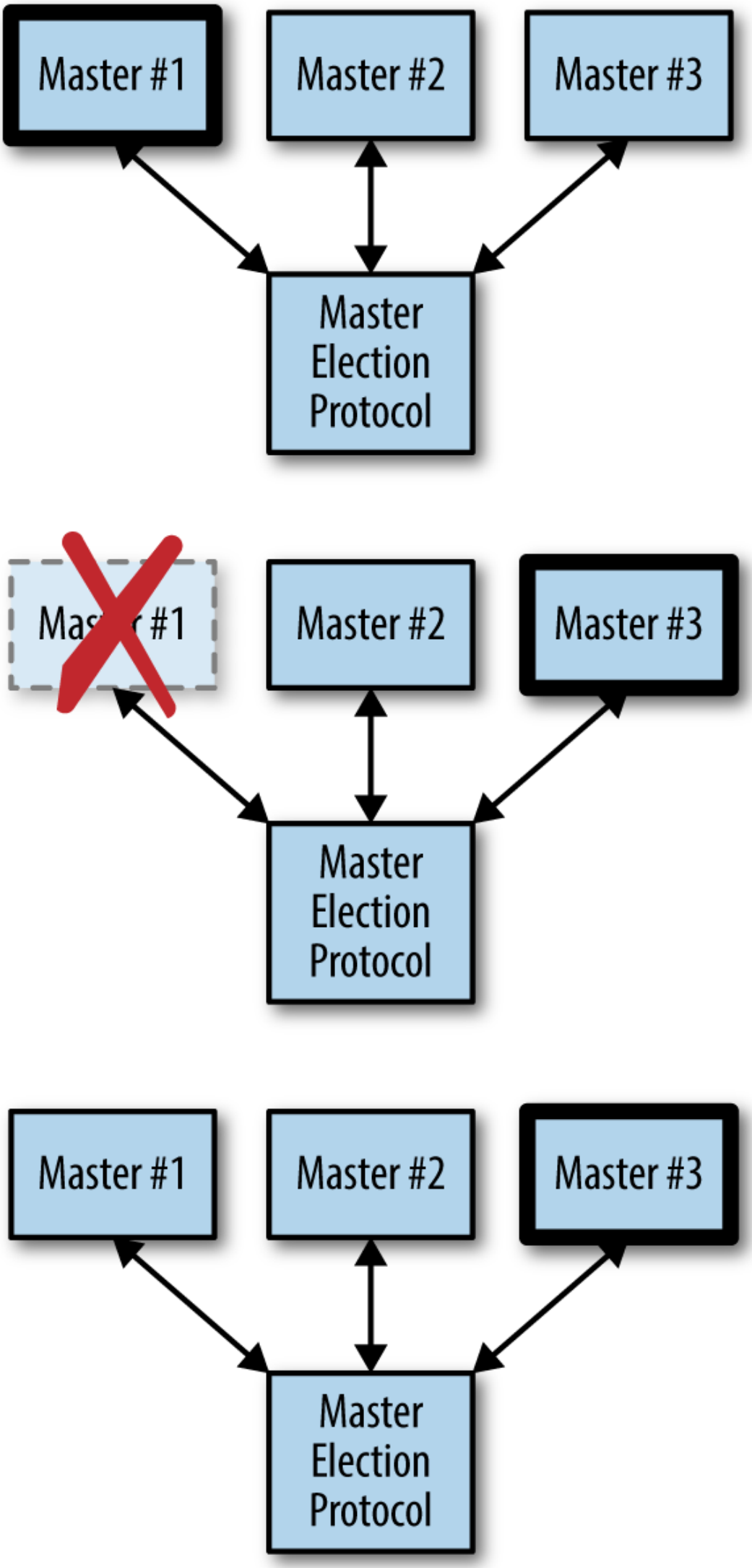
Master #1
Master #2
Master #3
Master Election Protocol
Master #2
Master #3
Master Election Protocol
Master #1
Master #2
Master #3
Master Election Protocol

Figure 9-1. A master election protocol in operation: initially the first master is selected, but when it fails, the third master takes over

Often, establishing distributed ownership is both the most complicated and most important part of designing a reliable distributed system.

Determining If You Even Need Master Election

The simplest form of ownership is to just have a single replica of the service. Since there is only one instance running at a time, that instance implicitly owns everything without any need for election. This has advantages of simplifying your application and deployment, but it has disadvantages in terms of downtime and reliability. However, for many applications, the simplicity of this singleton pattern may be worth the reliability trade-off. Let's look at this further.

Assuming that you run your singleton in a container orchestration system like Kubernetes, you have the following guarantees:

- If the container crashes, it will automatically be restarted
- If the container hangs, and you implement a health check, it will automatically be restarted
- If the machine fails, the container will be moved to a different machine

Because of these guarantees, a singleton of a service running in a container orchestrator has pretty good uptime. To take the definition of "pretty good" a little further, let's examine what happens in each of these failure modes. If the container process fails or the container hangs, your application will be restarted in a few seconds. Assuming your container crashes once a day, this is roughly three to four nines of uptime (2 seconds of downtime / day ~= 99.99% uptime). If your container crashes less often, it's even better than that. If your machine fails, it takes a while for Kubernetes to decide that the machine has failed and move it over to a different machine; let's assume that takes around 5 minutes. Given that, if every machine in your cluster fails every day, then your service will have two nines of uptime. And honestly, if every machine in your cluster fails every day, then you have way worse problems than the uptime of your master-elected service.

It's worth considering, of course, that there are more reasons for downtime than just failures. When you are rolling out new software, it takes time to download and start the new version. With a singleton, you cannot have both old and new versions running at the same time, so you will need to take down the old version for the duration of the upgrade, which may be several minutes if your image is large. Consequently, if you deploy daily and it takes 2 minutes to upgrade your software, you will only be able to run a two nines service, and if you deploy hourly, it won't even be a single nine service. Of course, there are ways that you can speed up your deployment by pre-pulling the new image onto the machine before you run the update. This can reduce the time it takes to deploy a new version to a few seconds, but the trade-off is added complexity, which was what we were trying to avoid in the first place.

Regardless, there are many applications (e.g., background asynchronous processing) where such an SLA is an acceptable trade-off for application simplicity. One of the key components of designing a distributed system is deciding when the "distributed" part is actually unnecessarily complex. But there are certainly situations where high availability (four+ nines) is a critical component of the application, and in such systems you need to run multiple replicas of the service, where only one replica is the designated owner. The design of these types of systems is described in the sections that follow.

The Basics of Master Election

Imagine that there is a service *Foo* with three replicas: *Foo-1*, *Foo-2*, and *Foo-3*. There is also some object *Bar* that must only be "owned" by one of the replicas (e.g., *Foo-1*) at a time. Often this replica is called the *master*, hence the term *master election* used to describe the process of how this master is selected as well as how a new master is selected if that master fails.

There are two ways to implement this master election. This first is to implement a distributed consensus algorithm like Paxos or RAFT, but the complexity of these algorithms make them beyond the scope of this book and not worthwhile to implement. Implementing one of these algorithms is akin to implementing locks on top of assembly code compare-and-swap instructions. It's an interesting exercise for an undergraduate computer

science course, but it is not something that is generally worth doing in practice.

Fortunately, there are a large number of distributed key-value stores that have implemented such consensus algorithms for you. At a general level, these systems provide a replicated, reliable data store and the primitives necessary to build more complicated locking and election abstractions on top. Examples of these distributed stores include etcd, ZooKeeper, and consul. The basic primitives that these systems provide is the ability to perform a compare-and-swap operation for a particular key. If you haven't seen compare-and-swap before, it is basically an atomic operation that looks like this:

```
var lock = sync.Mutex{}
var store = map[string]string{}

func compareAndSwap(key, nextValue, currentValue string) (bool, error) {
  lock.Lock()
  defer lock.Unlock()
  _, containsKey := store[key]
  if !containsKey {
    if len(currentValue) == 0 {
      store[key] = nextValue
      return true, nil
    }
    return false, fmt.Errorf("Expected value %s for key %s, but
    found empty", currentValue, key)
  }
  if store[key] == currentValue {
    store[key] = nextValue
    return true, nil
  }
  return false, nil
}
```

Compare-and-swap atomically writes a new value if the existing value matches the expected value. If the value doesn't match, it returns false. If the value doesn't exist and `currentValue` is not null, it returns an error.

In addition to compare-and-swap, the key-value stores allow you to set a time-to-live (TTL) for a key. Once the TTL expires, the key is set back to empty.

Put together, these functions are sufficient to implement a variety of distributed synchronization primitives.

Hands On: Deploying etcd

`etcd` is a distributed lock server developed by CoreOS. It is robust and proven in production at high scale, and is used by a variety of projects including Kubernetes.

Deploying etcd has fortunately become quite easy due to the development of two different open source projects:

- Helm: a Kubernetes package manager supported by Microsoft Azure
- The etcd operator developed by CoreOS

NOTE

Operators are an interesting topic being explored by CoreOS. An operator is an online program that runs inside your container orchestrator with the express purpose of running one or more applications. The operator is responsible for creating, scaling, and maintaining the successful operation of the program. Users configure the application through a desired state API. For example, the etcd operator is in charge of monitoring etcd itself. Operators are still a new idea but represent an important new direction in building reliable distributed systems.

To deploy the etcd operator for CoreOS, we're going to use the `helm` package management tool. Helm is an open source package manager that is part of the Kubernetes project, and was developed by Deis. Deis was acquired by Microsoft Azure in 2017 and Microsoft continues to support the further open source development of Helm.

If this is your first time using `helm`, you need to install the `helm` tool, following the instructions here: *https://github.com/kubernetes/helm/releases*.

Once you have the `helm` tool installed in your environment, you can install the etcd operator using `helm`, as follows:

```
# Initialize helm
helm init
```

```
# Install the etcd operator
helm install stable/etcd-operator
```

Once the operator is installed, it creates a custom Kubernetes resource to represent the etcd cluster. The operator is running, but there are no etcd clusters yet. To create an etcd cluster, you need to create a declarative configuration:

```
apiVersion: "etcd.coreos.com/v1beta1"
kind: "Cluster"
metadata:
  # Whatever name you want here
  name: "my-etcd-cluster"
spec:
  # 1, 3, 5 are the options for size
  size: 3
  # The version of etcd to install
  version: "3.1.0"
```

Save this configuration to *etcd-cluster.yaml* and then create the cluster using `kubectl create -f etcd-cluster.yaml`.

Creating this cluster will cause the the operator to create pods for the replicas of the `etcd` cluster. You can see the running replicas using:

```
kubectl get pods
```

Once all three replicas are running, you can get their endpoints using:

```
export ETCD_ENDPOINTS=kubectl get endpoints example-etcd-cluster
"-o=jsonpath={.subsets[*].addresses[*].ip}:2379,"
```

You can then store something into etcd using:

```
kubectl exec my-etcd-cluster-0000 -- sh -c "ETCD_API=3 etcdctl
--endpoints=${ETCD_ENDPOINTS} set foo bar"
```

Implementing Locks

The simplest form of synchronization is the mutual exclusion lock (aka Mutex). Anyone who has done concurrent programming on a single ma-

chine is familiar with locks, and the same concept can be applied to distributed replicas. Instead of local memory and assembly instructions, these distributed locks can be implemented in terms of the distributed key-value stores described previously.

As with locks in memory, the first step is to acquire the lock:

```
func (Lock l) simpleLock() boolean {
  // compare and swap "1" for "0"
  locked, _ = compareAndSwap(l.lockName, "1", "0")
  return locked
}
```

But of course, it's possible that the lock doesn't already exist, because we are the first to claim it, so we need to handle that case, too:

```
func (Lock l) simpleLock() boolean {
  // compare and swap "1" for "0"
  locked, error = compareAndSwap(l.lockName, "1", "0")
  // lock doesn't exist, try to write "1" with a previous value of
  // non-existent
  if error != nil {
    locked, _ = compareAndSwap(l.lockName, "1", nil)
  }
  return locked
}
```

Traditional lock implementations block until the lock is acquired, so we actually want something like this:

```
func (Lock l) lock() {
  while (!l.simpleLock()) {
    sleep(2)
  }
}
```

This implementation, though simple, has the problem that you will always wait at least a second after the lock is released before you acquire the lock. Fortunately, many key-value stores let you watch for changes instead of polling, so you can implement:

```
func (Lock l) lock() {
  while (!l.simpleLock()) {
    waitForChanges(l.lockName)
  }
}
```

Given this locking function, we can also implement unlock:

```
func (Lock l) unlock() {
  compareAndSwap(l.lockName, "0", "1")
}
```

You might now think that we are done, but remember that we are building this for a distributed system. A process could fail in the middle of holding the lock, and at that point there is no one left to release it. In such a situation, our system will become stuck. To resolve this, we take advantage of the TTL functionality of the key-value store. We change our `simpleLock` function so that it always writes with a TTL, so if we don't unlock within a given time, the lock will automatically unlock.

```
func (Lock l) simpleLock() boolean {
  // compare and swap "1" for "0"
  locked, error = compareAndSwap(l.lockName, "1", "0", l.ttl)
  // lock doesn't exist, try to write "1" with a previous value of
  // non-existent
  if error != nil {
    locked, _ = compareAndSwap(l.lockName, "1", nil, l.ttl)
  }
  return locked
}
```

NOTE

When using distributed locks, it is critical to ensure that any processing you do doesn't last longer than the TTL of the lock. One good practice is to set a watchdog timer when you acquire the lock. The watchdog contains an assertion that will crash your program if the TTL of the lock expires before you have called `unlock`.

By adding TTL to our locks, we have actually introduced a bug into our `unlock` function. Consider the following scenario:

1. Process-1 obtains the lock with TTL *t*.
2. Process-1 runs really slowly for some reason, for longer than *t*.
3. The lock expires.
4. Process-2 acquires the lock, since Process-1 has lost it due to TTL.
5. Process-1 finishes and calls *unlock*.
6. Process-3 acquires the lock.

At this point, Process-1 believes that it has unlocked the lock that it held at the beginning; it doesn't understand that it has actually lost the lock due to TTL, and in fact unlocked the lock held by Process-2. Then Process-3 comes along and also grabs the lock. Now both Process-2 and Process-3 both believe they own the lock, and hilarity ensues.

Fortunately, the key-value store provides a *resource version* for every write that is performed. Our lock function can store this resource version and augment `compareAndSwap` to ensure that not only is the value as expected, but the resource version is the same as when the lock operation occurred. This changes our simple `Lock` function to look like this:

```
func (Lock l) simpleLock() boolean {
  // compare and swap "1" for "0"
  locked, l.version, error = compareAndSwap(l.lockName, "1", "0", l.ttl)
  // lock doesn't exist, try to write "1" with a previous value of
  // non-existent
  if error != null {
    locked, l.version, _ = compareAndSwap(l.lockName, "1", null, l.ttl)
  }
  return locked
}
```

And the `unlock` function then looks like this:

```
func (Lock l) unlock() {
  compareAndSwap(l.lockName, "0", "1", l.version)
}
```

This ensures that the lock is only unlocked if the TTL has not expired.

Hands On: Implementing Locks in etcd

To implement locks in etcd, you can use a key as the name of the lock and pre-condition writes to ensure that only one lock holder is allowed at a

time. For simplicity, we'll use the `etcdctl` command line to lock and unlock the lock. In reality, of course, you would want to use a programming language; there are etcd clients for most popular programming languages.

Let's start by creating a lock named `my-lock`:

```
kubectl exec my-etcd-cluster-0000 -- sh -c \
  "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} set my-lock unlocked
```

This creates a key in etcd named `my-lock` and sets the initial value to `unlocked`.

Now let's suppose that Alice and Bob both want to take ownership of `my-lock`. Alice and Bob both try to write their name to the lock, using a precondition that the value of the lock is `unlocked`.

Alice first runs:

```
kubectl exec my-etcd-cluster-0000 -- sh -c \
  "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
      set --swap-with-value unlocked my-lock alice"
```

And obtains the lock. Now Bob attempts to obtain the lock:

```
kubectl exec my-etcd-cluster-0000 -- sh -c \
  "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
      set --swap-with-value unlocked my-lock bob"
Error:  101: Compare failed ([unlocked != alice]) [6]
```

You can see that Bob's attempt to claim the lock has failed, since Alice currently owns the lock.

To unlock the lock, Alice writes `unlocked` with a precondition value of `alice`:

```
kubectl exec my-etcd-cluster-0000 -- sh -c \
  "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
      set --swap-with-value alice my-lock unlocked"
```

Implementing Ownership

While locks are great for establishing temporary ownership of some critical component, sometimes you want to take ownership for the duration of the time that the component is running. For example, in a highly available deployment of Kubernetes, there are multiple replicas of the scheduler but only one replica is actively making scheduling decisions. Further, once it becomes the active scheduler, it remains the active scheduler until that process fails for some reason.

Obviously, one way to do this would be to extend the TTL for the lock to a very long period (say a week or longer), but this has the significant downside that if the current lock owner fails, a new lock owner wouldn't be chosen until the TTL expired a week later.

Instead, we need to create a *renewable lock*, which can be periodically renewed by the owner so that the lock can be retained for an arbitrary period of time.

We can extend the existing `Lock` that we defined previously to create a *renewable lock*, which enables the lock holder to renew the lock:

```
func (Lock l) renew() boolean {
  locked, _ = compareAndSwap(l.lockName, "1", "1", l.version, ttl)
  return locked
}
```

Of course, you probably want to do this repeatedly in a separate thread so that you hold onto the lock indefinitely. Notice that the lock is renewed every `ttl/2` seconds; that way there is significantly less risk that the lock will accidentally expire due to timing subtleties:

```
for {
  if !l.renew() {
    handleLockLost()
  }
  sleep(ttl/2)
}
```

Of course, you need to implement the `handleLockLost()` function so that it terminates all activity that required the lock in the first place. In a container orchestration system, the easiest way to do this may simply be

to terminate the application and let the orchestrator restart it. This is safe, because some other replica has grabbed the lock in the interim, and when the restarted application comes back online it will become a secondary listener waiting for the lock to become free.

Hands On: Implementing Leases in etcd

To see how we implement leases using etcd, we will return to our earlier locking example and add the `--ttl=<seconds>` flag to our lock create and update calls. The `ttl` flag defines a time after which the lock that we create is deleted. Because the lock disappears after the `ttl` expires, instead of creating with the value of *unlocked*, we will assume that the absence of the lock means that it is unlocked. To do this, we use the `mk` command instead of the `set` command. `etcdctl mk` only succeeds if the key does *not* currently exist.

Thus, to lock a leased lock, Alice executes:

```
kubectl exec my-etcd-cluster-0000 -- \
    sh -c "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
        --ttl=10 mk my-lock alice"
```

This creates a leased lock with a duration of 10 seconds.

For Alice to continue to hold the lock, she needs to execute:

```
kubectl exec my-etcd-cluster-0000 -- \
    sh -c "ETCD_API=3 etcdctl --endpoints=${ETCD_ENDPOINTS} \
        set --ttl=10 --swap-with-value alice my-lock alice"
```

It may seem odd that Alice is continually rewriting her own name into the lock, but this is the way the lock lease is extended beyond the 10-second TTL.

If, for some reason, the TTL expires, then the lock update will fail, and Alice will go back to creating the lock using the `etcd mk` command, or Bob may also use the `mk` command to obtain the lock for himself. Bob will likewise need to set and update the lock every 10 seconds to maintain ownership.

Handling Concurrent Data Manipulation

Even with all of the locking mechanisms we have described, it is still possible for two replicas to simultaneously believe they hold the lock for a very brief period of time. To understand how this can happen, imagine that the original lock holder becomes so overwhelmed that its processor stops running for minutes at a time. This can happen on extremely overscheduled machines. In such a case, the lock will time out and some other replica will own the lock. Now the processor frees up the replica that was the original lock holder. Obviously, the `handleLockLost()` function will quickly be called, but there will be a brief period where the replica still believes it holds the lock. Although such an event is fairly unlikely, systems need to be built to be robust to such occurrences. The first step to take is to double-check that the lock is still held, using a function like this:

```
func (Lock l) isLocked() boolean {
  return l.locked && l.lockTime + 0.75 * l.ttl > now()
}
```

If this function executes prior to any code that needs to be protected by a lock, then the probability of two masters being active is significantly reduced, but—it is important to note—it is not completely eliminated. The lock timeout could always occur between the time that the lock was checked and the guarded code was executed. To protect against these scenarios, the system that is being called from the replica needs to validate that the replica sending a request is actually still the master. To do this, the hostname of the replica holding the lock is stored in the key-value store in addition to the state of the lock. That way, others can double-check that a replica asserting that it is the master is in fact the master.

This system diagram is shown in Figure 9-2. In the image, `shard2` is the owner of the lock, and when a request is sent to the worker, the worker double-checks with the lock server and validates that `shard2` is actually the current owner.

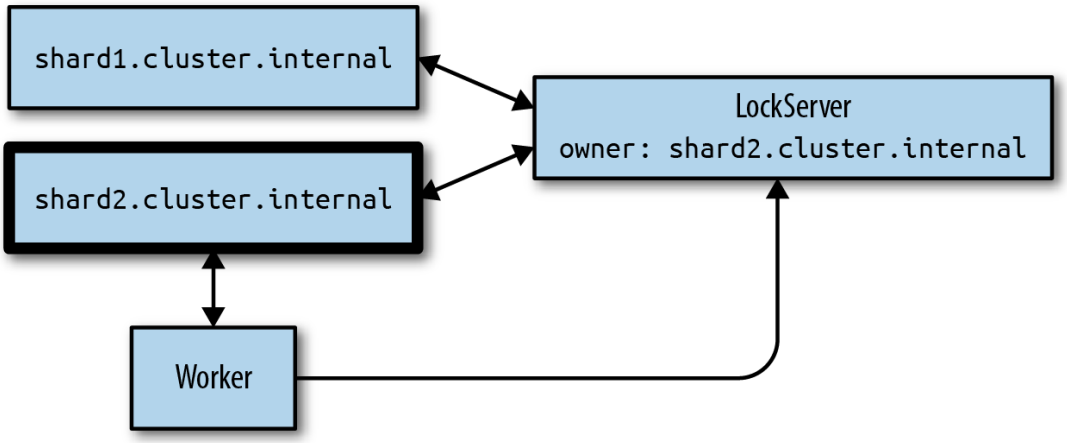

Figure 9-2. A worker double-checking to validate that the requester who sent a message is actually the current owner of the shard

In the second case, `shard2` has lost ownership of the lock, but it has not yet realized this so it continues to send requests to the worker node. This time, when the worker node receives a request from `shard2`, it double-checks with the lock service and realizes that `shard2` is no longer the lock owner, and thus the requests are rejected.

To add one final further complicating wrinkle, it's always possible that ownership could be obtained, lost, and then re-obtained by the system, which could actually cause a request to succeed when it should actually be rejected. To understand how this is possible, consider the following sequence of events:

1. Shard-1 obtains ownership to become master.
2. Shard-1 sends a request `R1` as master at time `T1`.
3. The network hiccups and delivery of `R1` is delayed.
4. Shard-1 fails TTL because of the network and loses lock to Shard-2.
5. Shard-2 becomes master and sends a request `R2` at time `T2`.
6. Request `R2` is received and processed.
7. Shard-2 crashes and loses ownership back to Shard-1.
8. Request `R1` finally arrives, and Shard-1 is the current master, so it is accepted, but this is bad because `R2` has already been processed.

Such sequences of events seem byzantine, but in reality, in any large system they occur with disturbing frequency. Fortunately, this is similar to the case described previously, which we resolved with the resource version in etcd. We can do the same thing here. In addition to storing the name of the current owner in etcd, we also send the resource version along with each request. So in the previous example, `R1` becomes `(R1, Version1)`. Now when the request is received, the double-check vali-

dates both the current owner *and* the resource version of the request. If either match fails, the request is rejected. This patches up this example.